Technische Universität Berlin
Faculty IV: Electrical Engineering and Computer Science
Freie Universität Berlin
Department of Mathematics and Computer Science
Algorithmic Bioinformatics

welche uni angeben, welche reihenfolge..?

Automatic Optimization of Level Parameters for RSHash

(RSHash is a minimizer based fast, compressed hash table for storing DNA k-mers. It was recently introduced by the Reinert Group.)

Bjarne David Jehn

Thesis submitted in fulfillment of the requirements for the degree "Bachelor of Science"(B. Sc.) in the field of Computer Science

First Reviewer: Prof. Dr. Knut Reinert
Second Reviewer: Prof. Dr. Stephan Kreutzer
Supervisor: Jonas Schulte-Mattler

Declaration of Authorship

I hereby declare that the thesis submitted is my own, unaided work, completed without any external help. Only the sources and resources listed were used. All passages taken from the sources and aids used, either unchanged or paraphrased, have been marked as such.

Where generative AI tools were used, I have indicated the product name, manufacturer, the software version used, as well as the respective purpose (e.g. checking and improving language in the texts, systematic research). I am fully responsible for the selection, adoption, and all results of the AI-generated output I use.

I have taken note of the Principles for Ensuring Good Research Practice at TU Berlin dated 15 February 2023. https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMBl/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf

I further declare that I have not submitted the thesis in the same or similar form to any other examination authority.

Berlin, _____ _____

Date Signature

Abstract

in English

Zusammenfassung

auf Deutsch

Contents

1 Introduction **5**
- 1.1 RSHash Explanation . . . 5
- 1.2 Problem Definition . . . 5

2 Minimizer Distribution **7**
- 2.1 Minimizer Distribution for Several Levels . . . 7

3 MINLP Modeling **8**
- 3.1 Notation . . . 8
- 3.2 One Level . . . 9
  - 3.2.1 Runtime . . . 9
  - 3.2.2 Space Consumption . . . 10
  - 3.2.3 Optimization Goal . . . 11
- 3.3 Several Levels . . . 13
  - 3.3.1 Runtime . . . 13
  - 3.3.2 Space Consumption . . . 14
- 3.4 Limitations . . . 14

4 Subsampling **16**
- 4.1 Motivation . . . 16
- 4.2 Subsampling Method . . . 16
- 4.3 Time Complexity of Finding the Minimizer Distributions . . . 16
- 4.4 Time Complexity using Subsampling . . . 16
- 4.5 Error Analysis . . . 17
- 4.6 Choosing l . . . 17
- 4.7 Choosing n . . . 17

5 Experiments **18**
- 5.1 Times for Operations on Datastructure . . . 18

Bibliography **19**

1 Introduction

1.1 RSHash Explanation

Wie detailiert soll ich RSHash beschreiben? Reicht eine grobe Beschreibung und ein Verweis auf [SMR26]?

RSHash is a minimizer based fast, compressed hash table for storing DNA k-mers that answers lookup and can be extended for supporting locate. It was recently introduced by the Reinert Group.
The data structure works with one, two or three levels and a hash table. For each level l a pair of level parameters has to be specified: the minimizer length m_l and a threshold t_l.
Each level l works in the following way: The minimizer of length m_l is computed for each super-k-mer in the input whilst also counting the frequency (occurrences) of all appearing minimizers.
All super-k-mers with minimizers whose frequency does not succeed t_l are marked in a bit vector R, their offsets in the Text are stored in a vector O and a bit vector S marks the boundaries of the offsets for each minimizer.
Those super-k-mers with minimizers of a frequency larger than t_l make up the input for the next higher level $l + 1$ which works exactly like the previous apart from having its own level parameters.
Only k-mers with frequent minimizers on all specified levels end up being stored in the hash table.[SMR26].

1.2 Problem Definition

Choosing the minimizer length **m** is an inherent problem of minimizer based k-mer indexing methods. A minimizer is expected to be valid for an average of $(k - m + 2)/2$ k-mers. Using a small **m** thus leads to a relatively small number of super-k-mers meaning a better compression. Picking a small **m** however also means that minimizers will be more frequent which leads to high query times because each minimizer offset has to be evaluated for an actual match with the queried k-mer. This means a constant trade off between space consumption and query times. Existing methods use a number of approaches to deal with this problem

beispiele und quellen angeben

The leveled approach used by RSHash exploits the skewed distribution of minimizers.

It allows to achieve high compression by using relatively short minimizers on the lower levels whilst also mitigating the burden of very frequent minimizers by using thresholds to move them to higher levels.

So far the level parameters have to be specified when building the data structure which requires manual testing and extensive understanding of the RSHash algorithm.

With this thesis we attempt to expand the understanding of how exactly the level parameters influence the time and space consumption of RSHash and make the algorithm more accessible to use by automating the optimization of level parameters for a given genome and optimization goal.

Problems:

- work towards a better understanding of minimizer distribution across several levels
- analyze how level parameters influence time and space consumption of RSHash
- automize finding optimal level parameters

2 Minimizer Distribution

The skewness of minimizer distributions has been observed by several previous works [Chi+14; Jai+20; Pib22].
We will focus on the percentages $\mathbf{p_M}$ of minimizers with frequency at most f by minimizer length m and $\mathbf{p_K}$ of k-mers having those minimizers as described in [SMR26].

und > 0?

soll ich hier tatsächlich auch p_M einführen, obwohl ich p_M bisher nirgends verwende? ich denke schon, da p_M hilfreich ist, um p_K zu verstehen

We aim to analyze how the minimizer distribution behaves across the several levels used by RSHash in order to gain a better understanding of how to chose the level parameters.

2.1 Minimizer Distribution for Several Levels

The distribution of minimizers on higher levels is dependent on the choice of the level parameters off the levels below, as those influence which k-mers appear on higher levels. In order to understand this relationship, we will measure these distributions for a selection of sequences and analyze the results. It should be noted that this alone is not sufficient to argue about the general case.

3 MINLP Modeling

We introduce a Mixed Integer Non Linear Programming (MINLP) aiming to optimize the level parameters. In Section 3.2 we model the runtime and space needed by RSHash on one level in dependence of the level parameters and minimizers distribution. We then extend this to the general case of several levels in Section 3.3.
In Chapter 5 we show how ..... for a given Sequence we can then compute the minimizer distribution as outlined in Chapter 2 and use a MINLP solver like SCIP [Ach+08; Ach09; Bes+23] to find those level parameters leading to the optimization goal.

..

3.1 Notation

Let q be a k-mer. We refer to its minimizer $m(q)$ and the minimizers frequency $f(m(q))$.
We denote the time needed to perform an *operation* on a *data structure* with $T_{operation_{data\ structure}}$ - e.g. the time for a rank query on the bit vector R is T_{rank_R}.
We use $\mathcal{M}$ and $\mathcal{T}$ to denote the sets of minimizer lengths and thresholds considered when searching for optimal parameters.
We denote the optimal minimizer length and threshold by $\mathbf{m}$ and $\mathbf{t}$.
Closely following the notation introduced by [SMR26] $\mathbf{p_K}$ denotes the percentage of appearing k-mers q with infrequent minimizers of length $\mathbf{m}$ ($f(m(q)) \leq \mathbf{t}$). $p_K(m,t)$ on the other hand is the empirically measured percentage on a fixed sequence and a given pair $(m,t) \in \mathcal{M} \times \mathcal{T}$.
Similarly $\mathbf{f_{avg}}$ describes the average frequency of all infrequent minimizers when using $\mathbf{m}, \mathbf{t}$, whilst $f_{avg}(m,t)$ is the empirically measured average frequency for a specific sequence and a given pair $(m,t) \in \mathcal{M} \times \mathcal{T}$. $f_{avg}(m,t)$ can be derived from $p_K(m,t)$:

model these times as part of the optimization OR measure in advance?

$$
\begin{aligned}
f_{avg}(m,t) &= \sum_{f=1}^{t} \frac{(p_K(m,f) - p_K(m,f-1))}{p_K(m,t)} \cdot f \\
&= \frac{1}{p_K(m,t)} \cdot \sum_{f=1}^{t} (p_K(m,f) - p_K(m,f-1)) \cdot f
\end{aligned}
$$

tabelle $f_avg(m,t)$ mittels dynamischer programmierung bauen? - eigentlich nicht nötig, wenn distinct

3.2 One Level

3.2.1 Runtime

Lookup

In order to model the time $T_{lookup_{RSHash}}$ needed for a lookup using RSHash, we closely follow the lookup procedure as described by [SMR26]:
A k-mer q is looked up by first computing its minimizer $m(q)$ and checking whether the bit $R[m(q)]$ is set.
For the $\mathbf{p_K}$ of the k-mers for which the bit is set, the minimizer's rank $r = rank_R(m(q))$ is computed. Then $sel_S(r)$ and $sel_S(r+1) - 1$ tell the indices in Offsets O where the minimizer's first and last offset in the text T are stored.
Finally the presence of q in T is checked at every offset $o \in O[sel_S(r) : sel_S(r+1) - 1]$ until q is either found or the end of the interval is reached.
This offset interval contains an average of $\mathbf{f_{avg}}$ offsets. In the lookup case, the comparison is terminated as soon as q is found for the first time, which is on average the case after $(\mathbf{f_{avg}} + 1)/2$ comparisons (+1 is necessary because the first index always has to be checked).
For $1 - \mathbf{p_K}$ $m(q)$ does not appear at all or has a $f(m(q)) > t$. In this case the lookup is completed through a lookup of q in the hash table.

The lookup time $T_{lookup_{RSHash}}$ for one level can thus be modeled as:

$$\begin{aligned} T_{lookup_{RSHash}} = T_{acc_R} + \mathbf{p_K} \cdot (T_{rank_R} + 2T_{sel_S} + (\mathbf{f_{avg}} + 1)/2 \cdot (T_{acc_O} + T_{comp_T})) \\ + (1 - \mathbf{p_K}) \cdot T_{lookup_H} \end{aligned} \tag{3.1}$$

In order to determine $\mathbf{m}$ and $\mathbf{t}$ we introduce binary selector variables μ_m, τ_t for each possible $m \in \mathcal{M}$, $t \in \mathcal{T}$. μ_m should be 1 exactly when the optimal minimizer length is m and τ_t should be 1 exactly when the optimal threshold is t. This is ensured through:

$$\mathbf{m} = \sum_{m \in \mathcal{M}} \mu_m \cdot m \tag{3.2}$$

$$\mathbf{t} = \sum_{t \in \mathcal{T}} \tau_t \cdot t \tag{3.3}$$

The following constraints force the selectors for $\mathbf{m}$ and $\mathbf{t}$ to be binary and mutually exclusive:

$$\mu_m \in \{0, 1\} \mid \forall m \in \mathcal{M} \tag{3.4}$$

$$\tau_t \in \{0, 1\} \mid \forall t \in \mathcal{T} \tag{3.5}$$

$$\sum_{m \in \mathcal{M}} \mu_m = 1 \tag{3.6}$$

$$\sum_{t \in \mathcal{T}} \tau_t = 1 \tag{3.7}$$

With $p_K(m,t)$ being the empirically determined percentage of k-mers q with $1 \leq f(m(q)) \leq t$ (for a fixed $(m,t) \in \mathcal{M} \times \mathcal{T}$) we can then model a constraint forcing $\mathbf{p_K}$ to take the value of the only $p_K(m,t)$ for which $\mu_m = 1$ and $\tau_t = 1$:

$$\mathbf{p_K} = \sum_{m \in \mathcal{M}} \left(\mu_m \sum_{t \in \mathcal{T}} p_K(m,t) \cdot \tau_t \right) \tag{3.8}$$

Analogously with $f_{avg}(m,t)$ being the empirically determined average frequency of k-mers q with $1 \leq f(m(q)) \leq t$ (for a fixed $(m,t) \in \mathcal{M} \times \mathcal{T}$) we can then model $\mathbf{f_{avg}}$ with:

$$\mathbf{f_{avg}} = \sum_{m \in \mathcal{M}} \left(\mu_m \sum_{t \in \mathcal{T}} f_{avg}(m,t) \cdot \tau_t \right) \tag{3.9}$$

Without further constraints, solving the described MINLP for minimal $T_{lookup_{RSHash}}$ results in a solution where the μ_m, τ_t corresponding to the smallest $m \in \mathcal{M}$, $t \in \mathcal{T}$ will be set to one.
This is expected, as this leads to the smallest possible $\mathbf{p_K}$ which minimizes the k-mers having to be looked up in the text and maximizes the lookups in the (faster) hash table. This however results in maximal space consumption, as each entry in the hash table is significantly larger than an entry in O as described in Section 1.2.
In order to account for that, we have to add constraints modeling the space consumption.

3.2.2 Space Consumption

The space consumption of RSHash can be decomposed into the space needed for the data structures components: the text T, the vectors R, S and O as well as the hash table H. Instead of modeling the total space used by RSHash we will focus on the bits per k-mer (bpk) used by the data structure as this metric allows to compare compression independent of the inputs length.
We use that using Elias-Fano Coding, bit vector of length U with n 1 s can be encoded with $n\lceil log(U/n) \rceil + 3n$ many bits [OS07]

The text T is much longer then k. Because of this the bits per k-mer for storing the text T can be be approximated to be 1:

$$bpk_T = 1 \tag{3.10}$$

model or measure bpk_R and bpk_S??

Bit vector R is a bit vector of length $4^{\mathbf{m}}$ that contains $n_{minimizers}$ 1s marking the appearance of each encountered minimizer in the sequences k-mers. Using Elias-Fano-Encoding, its size can be compressed to $n_{minimizers}\lceil log(4^{\mathbf{m}}/n_{minimizers}) \rceil + 3n_{minimizers}$ [OS07]. For a total number of k-mers n_{kmers} the bits per k-mer of R are:

$$bpk_R = \frac{1}{n_{kmers}} \cdot \left(n_{minimizers} \lceil log \frac{4^{\mathbf{m}}}{n_{minimizers}} \rceil + 3n_{minimizers} \right) \tag{3.11}$$

the code for computing the minimizer stats already uses n - it could easily be used

define

Bit vector S has length $|O|+1 = n_{skmers}+1$. Each minimizer appears $\mathbf{f_{avg}}$ times on average, which means that the bit vector contains $(n_{skmers}+1)/\mathbf{f_{avg}}$ 1s marking the the boundaries of the offsets for each minimizers. Using Elias-Fano-Encoding, its size can be compressed to $((n_{skmers}+1)/\mathbf{f_{avg}})\lceil log((n_{skmers}+1)/((n_{skmers}+1)/\mathbf{f_{avg}}))\rceil + 3((n_{skmers}+1)/\mathbf{f_{avg}})$ [OS07]. For a total number of kmers n_{kmers} the bits per k-mer of S are:

$$\begin{aligned} bpk_S &= \frac{1}{n_{kmers}} \cdot \left(\frac{n_{skmers}+1}{\mathbf{f_{avg}}} \lceil log \frac{n_{skmers}+1}{\frac{n_{skmers}+1}{\mathbf{f_{avg}}}} \rceil + 3\frac{n_{skmers}+1}{\mathbf{f_{avg}}} \right) \\ &= \frac{1}{n_{kmers}} \cdot \left(\frac{n_{skmers}+1}{\mathbf{f_{avg}}} \lceil log \mathbf{f_{avg}} \rceil + 3\frac{n_{skmers}+1}{\mathbf{f_{avg}}} \right) \end{aligned} \tag{3.12}$$

The offset vector O contains one entry of size $log|T|$ for the offset of each super-k-mer having an infrequent minimizer.
Since $\mathbf{p_K}$ describes the percentage of k-mers with infrequent minimizers and every minimizer covers an average of $(k-\mathbf{m}+2)/2$ consecutive k-mers, we can model the bits per k-mer in O as:

$$bpk_O = \frac{\mathbf{p_K}}{(k-\mathbf{m}+2)/2} \cdot log|T| \tag{3.13}$$

The hash table H contains an entry $(key : q, value : offset(q))$ for $1-\mathbf{p_K}$ of the k-mers q in T. Each entry has size $size(key)+size(value) = 64+log|T|$. We introduce the load factor α to the model, in order to account for empty buckets in the hash table. The bits per k-mer required for the hash table can thus be modeled by

$$bpk_H = (1-\mathbf{p_K}) \cdot \frac{(64+log|T|)}{\alpha} \tag{3.14}$$

The total bits per k-mer can be modeled by simply adding up the space requirements of each component:

$$bpk_{total} = bpk_T + bpk_R + bpk_S + bpk_O + bpk_H \tag{3.15}$$

reminder to include somewhere that we set $\alpha = 0.75$ in practice and explain why that is a reasonable value

3.2.3 Optimization Goal

Having defined all constraints regarding the lookup time and space consumption of RSHash, we can now build a MINLP in order to find the optimal level parameters.
The sets of possible minimizer lengths $\mathcal{M}$ and thresholds $\mathcal{T}$, the measured minimizer statistics p_K and f_{avg} for those sets as well as the times needed for the access, rank and select queries on the data structures different components are passed as arguments to the MINLP.
We introduce three more arguments: The optimization goal *OptGoal* which specifies whether to minimize $T_{lookup_{RSHash}}$ or bpk_{total} and upper bounds $Bound_{time}, Bound_{bpk}$ for the lookup time and space consumption.

$$OptGoal \in \{T_{lookup_{RSHash}}, bpk_{total}\} \tag{3.16}$$
$$Bound_{time} \in \mathbb{R}_0^+ \geq 0 \tag{3.17}$$
$$Bound_{bpk} \in \mathbb{R}_0^+ \geq 0 \tag{3.18}$$

Figure 3.1: Complete MINLP For One Level

Input: $OptGoal, Bound_{time}, Bound_{bpk}, \mathcal{M}, \mathcal{T}, p_K, f_{avg}, T_{acc_R}, T_{rank_R}, T_{sel_S}, T_{acc_O}, T_{comp_T}, T_{lookup_H}$

$$
\begin{aligned}
&\text{MINIMIZE} && \\
&& OptGoal \\
&\text{SUBJECT TO} && \\
& T_{lookup_{RSHash}} && \leq Bound_{time} \\
& T_{lookup_{RSHash}} && = T_{acc_R} + \mathbf{p_K} \cdot (T_{rank_R} + 2T_{sel_S} + (\mathbf{f_{avg}} + 1)/2 \cdot (T_{acc_O} + T_{comp_T})) \\
& && \quad + (1 - \mathbf{p_K}) \cdot T_{lookup_H} \\
& \mathbf{p_K} && = \sum_{m \in \mathcal{M}} \left(\mu_m \sum_{t \in \mathcal{T}} p_K(m,t) \cdot \tau_t \right) \\
& \mathbf{f_{avg}} && = \sum_{m \in \mathcal{M}} \left(\mu_m \sum_{t \in \mathcal{T}} f_{avg}(m,t) \cdot \tau_t \right) \\
& \mathbf{m} && = \sum_{m \in \mathcal{M}} \mu_m \cdot m \\
& \mathbf{t} && = \sum_{t \in \mathcal{T}} \tau_t \cdot t \\
& \sum_{m \in M} \mu_m && = 1 \\
& \sum_{t \in T} \tau_t && = 1 \\
& \mu_m && \in \{0,1\} \mid \forall m \in M \\
& \tau_t && \in \{0,1\} \mid \forall t \in T \\
& bpk_{total} && \leq Bound_{bpk} \\
& bpk_{total} && = bpk_T + bpk_R + bpk_S + bpk_O + bpk_H \\
& bpk_T && = 1 \\
& bpk_R && = ?? \\
& bpk_S && = ?? \\
& bpk_O && = \frac{\mathbf{p_K}}{(k - \mathbf{m} + 2)/2} \cdot log|T| \\
& bpk_H && = (1 - \mathbf{p_K}) \cdot \frac{(64 + log|T|)}{\alpha}
\end{aligned}
$$

When using the MINLP, the upper bound belonging to the optimization goal should be set to ∞. In addition to the already defined constraints, $T_{lookup_{RSHash}} \leq Bound_{time}$ and $bpk_{total} \leq Bound_{bpk}$ are added to the MINLP to enforce these bounds. Figure 3.1 shows the constructed MINLP.

One could for example search for level parameters minimizing the lookup time whilst not exceeding eight bits per k-mer by setting $OptGoal \leftarrow T_{lookup_{RSHash}}, Bound_{time} \leftarrow \infty, Bound_{bpk} \leftarrow 8$.

add equation references

3.3 Several Levels

In the following, we generalize the constraints defined in Section 3.2 to model the use of $L \geq 1$ levels. We extend the Section 3.1 by adding a superscript l in order to indicate an instance belonging to level $1 \leq l \leq L$. We still assume, that the times $T_{acc_{R^l}}, T_{rank_{R^l}}, T_{sel_{S^l}}, T_{acc_{O^l}}, T_{comp_T}, T_{lookup_H}$ are (practically) independent of the parameters $\mathbf{m}^l, \mathbf{t}^l$ and the level l. reason

3.3.1 Runtime

When using L levels, the probability of a k-mer belonging to a higher level is dependent on the probability of k-mers belonging to the levels below.
For $\mathbf{p}_\mathbf{K}^l$ of the k-mers on a level $1 \leq l \leq L$ the lookup is completed by a search through the offsets on that level whilst the remaining $(1 - \mathbf{p}_\mathbf{K}^l)$ of queries result in a lookup on the next level.
We can thus model the lookup time for all levels $1 \leq l \leq L$ recursively as:

$$T^l_{lookup_{RSHash}} = \begin{cases} T_{acc_{R^l}} + \mathbf{p}_\mathbf{K}^l \cdot (T_{rank_{R^l}} + 2T_{sel_{S^l}} + \dfrac{\mathbf{f}^l_\mathbf{avg} + 1}{2} \cdot (T_{acc_{O^l}} + T_{comp_T})) & \mid 1 \leq l \leq L \\ \qquad + (1 - \mathbf{p}_\mathbf{K}^l) \cdot T^{l+1}_{lookup_{RSHash}} & \\ T_{lookup_H} & \mid l = L + 1 \end{cases}$$

As an example for $L = 2$ levels, this results in:

$$\begin{aligned} T_{lookup_{RSHash}} = T_{acc_{R^1}} &+ \mathbf{p}_\mathbf{K}^1 \cdot (T_{rank_{R^1}} + 2T_{sel_{S^1}} + \frac{\mathbf{f}^1_\mathbf{avg} + 1}{2} \cdot (T_{acc_{O^1}} + T_{comp_T})) \\ &+ (1 - \mathbf{p}_\mathbf{K}^1) \cdot \Bigg(T_{acc_{R^2}} + \mathbf{p}_\mathbf{K}^2 \cdot (T_{rank_{R^2}} + 2T_{sel_{S^2}} + \frac{\mathbf{f}^2_\mathbf{avg} + 1}{2} \cdot (T_{acc_{O^2}} + T_{comp_T})) \\ &\qquad + (1 - \mathbf{p}_\mathbf{K}^2) \cdot T_{lookup_H} \Bigg) \end{aligned}$$

Unique sets of binary selector variables μ^l_m and τ^l are needed for the level parameters $\mathbf{m}^l$ and $\mathbf{t}^l$ of each level l. The constraints regarding these variables are analog to those when using only one level:

$$\begin{aligned} \mathbf{m}^l &= \sum_{m \in \mathcal{M}} \mu^l_m \cdot m^l && \mid \forall l \\ \mathbf{t}^l &= \sum_{t \in \mathcal{T}} \tau^l_t \cdot t^l && \mid \forall l \\ \mu^l_m &\in \{0, 1\} && \mid \forall (l, m) \in (\{1, ..., L\} \times \mathcal{M}) \\ \tau^l_t &\in \{0, 1\} && \mid \forall (l, t) \in (\{1, ..., L\} \times \mathbf{t}) \\ \sum_{m \in \mathcal{M}} \mu^l_m &= 1 && \mid \forall l \\ \sum_{t \in \mathcal{T}} \tau^l_t &= 1 && \mid \forall l \end{aligned}$$

For each level l $\mathbf{p_K^l}$ and $\mathbf{f_{avg}^l}$ can also be modeled analogously to using only one level by making use of the Section 2.1:

$$\mathbf{p_K^l} = \sum_{m \in \mathcal{M}} \left(\mu_m^l \sum_{t \in \mathcal{T}} p_K^l(m,t) \cdot \tau_t^l \right) \qquad | \, \forall l \in \{1, ..., L\}$$

$$\mathbf{f_{avg}^l} = \sum_{m \in \mathcal{M}} \left(\mu_m^l \sum_{t \in \mathcal{T}} f_{avg}^l(m,t) \cdot \tau_t^l \right) \qquad | \, \forall l \in \{1, ..., L\}$$

3.3.2 Space Consumption

The text T is only being stored once, so $bpk_T \approx 1$ still holds.

Bit vectors R^l **and** S^l ....

The offset vector O^l on level l only contains an entry for a k-mer q, if the k-mers minimizer is frequent on all previous levels but infrequent on level l. The bits per k-mer for the Offset vector on level l can thus be modeled as:

$$bpk_{O^l} = \prod_{i=1}^{l-1}(1 - \mathbf{p_K^i}) \cdot \frac{\mathbf{p_K^l}}{(k - \mathbf{m}^l + 2)/2} \cdot log|T| \mid \forall l \in \{1, ..., L\}$$

The hash table H only contains an entry for k-mers q, which have frequent minimizers on all levels. The bits per k-mer for the hash table can thus be modeled as

$$bpk_H = \prod_{l=1}^{L}(1 - \mathbf{p_K^l}) \cdot \frac{(64 + log|T|)}{\alpha}$$

The total bits per k-mer for L levels only differ to those on one level, as there are now a vector R, S and O on each level:

$$bpk_{total} = bpk_T + \sum_{l=1}^{L} (bpk_{R^l} + bpk_{S^l} + bpk_{O^l}) + bpk_H$$

3.4 Limitations

The MINLP approach for finding optimal level parameters has the advantage of finding the optimal level parameters for the specified optimization goal.

rather: should have the advantage... - wie kann ich die Korrektheit des MINLP Zeigen/Argumentieren??

However, finding the empirical values $p_K^l(m,t)$ for each level becomes increasingly time consuming for large genomes and more levels used as outlined in Section 2.1. This renders the approach implausible.

noch nicht geschrieben

For this reason, an approach yielding only an approximation of the optimal level parameters but taking less time to do so could be of interest.

We will further explore the method of Chapter 4 the genome in order to reduce the computation of minimizer distribution.

TODO

In another approach we will try to approximate the distributions on higher levels based on that on the first level in order to get arround the exponential cost of computing those values.

is it expo-nential?

TODO

4 Subsampling

4.1 Motivation

The time for processing a genome in order to find the minimizer distribution $\mathbf{p_K}(m, t)$ for possible minimizer lengths $m \in \mathcal{M}$ and thresholds $t \in \mathcal{T}$ is directly proportional to the size of the genome.
For large genomes, this processing can become the dominating factor when searching for optimal parameters using the Chapter 3 approach. In order to speed up the search for the empirical values, we introduce a subsampling method.

4.2 Subsampling Method

Instead of calculating the distribution over the full genome, we sample n sequences of length $l \geq k$ at random positions in the genome.

how to handle overlapping samples

4.3 Time Complexity of Finding the Minimizer Distributions

genaue zeitliche abhängigkeit 1. verstehen und 2. aufschreiben

4.4 Time Complexity using Subsampling

By applying the proposed subsampling strategy, the original genome T of length $|T|$ is reduced to the subsampled text T^S of length $|T^S| = n \cdot l$. This means a reduction in size by a factor of $\frac{|T|}{|T^S|}$.
Depending on the choice of n and l this results in a speedup of the processing time by a factor of or even a constant processing time when both parameters are chosen as a constand regardless of $|T|$ (this however should result in a growing error for larger genomes).

assuming no overlaps

factor

time

4.5 Error Analysis

4.6 Choosing l

The length l has to be at least k.

wenn $l > k$ ist, überlappen sich mehrere kmere an den gesampleten stellen.
Frage: ich verstehe noch nicht, warum genau dass von vorteil ist.
Antwort: Weil roling hash - l ca 1000, dann quasi linear

4.7 Choosing n

Choosing a constant n would result in a processing time that would no longer be dependent of the genomes length $|T|$, as the sampled text would have constant size $n \cdot l$. For large genomes, the percentage $\frac{|T^S|}{|T|}$ of the genome represented in the subsample can become arbitrarily small, resulting in growing error.
If instead n is chosen relative to $|T|$, $\frac{|T^S|}{|T|}$ will have a constant value independent of the genomes size. Even though this means, that the time needed, in order to find the minimizer distributions would still be proportional to the genomes size, this should be the preferred option as it results in a bounded error of the approximation.
For a large enough n, the probability of choosing a bad representation becomes highly unlikely.

assuming no overlaps

zu vage, begründen

5 Experiments

5.1 Times for Operations on Datastructure

incomplete. measure or model?

Vector Access Accessing an element of a Vector can be done in constant time, we use the following values based on measurements:

$$T_{acc_R} = ...$$
$$T_{acc_O} = ...$$

Hash Table Access Accessing an element of a Hash Table can be done in constant time, we use the following values based on measurements:

$$T_{lookup_H} = ...$$

Bit Vector Rank Query A bitvector of length U with n $1s$ can be encoded with $n\lceil logU/n \rceil + 3n$ many bits supporting rank in $O(log(U/n)$ time (Okanohara and Sadakane, 2007).

does that help here? Doesn't the model require actual time instead of O notation - maybe measurements would be more suited

Bit Vector Select Query ???
Comparison with Text ???

Bibliography

[Ach+08] T. Achterberg, T. Berthold, T. Koch, and K. Wolter. *Constraint Integer Programming: a New Approach to Integrate CP and MIP*. eng. Tech. rep. 08-01. Takustr. 7, 14195 Berlin: ZIB, 2008 (cit. on p. 8).

[Ach09] T. Achterberg. "SCIP: solving constraint integer programs". In: *Mathematical Programming Computation* 1.1 (2009), pp. 1–41. URL: https://doi.org/10.1007/s12532-008-0001-1 (cit. on p. 8).

[Bes+23] K. Bestuzheva, A. Chmiela, B. MÃ¼ller, F. Serrano, S. Vigerske, and F. Wegscheider. *Global Optimization of Mixed-Integer Nonlinear Programs with SCIP 8*. Tech. rep. 2301.00587. arXiv, 2023 (cit. on p. 8).

[Chi+14] R. Chikhi, A. Limasset, S. Jackman, J. Simpson, and P. Medvedev. *On the representation of de Bruijn graphs*. en. arXiv:1401.5383 [q-bio.QM]. Oct. 2014. URL: http://arxiv.org/abs/1401.5383 (visited on 09/18/2026) (cit. on p. 7).

[Jai+20] C. Jain, A. Rhie, H. Zhang, C. Chu, B. P. Walenz, S. Koren, and A. M. Phillippy. "Weighted minimizer sampling improves long read mapping". en. In: *Bioinformatics* 36.Supplement_1 (July 2020), pp. i111–i118. URL: https://academic.oup.com/bioinformatics/article/36/Supplement_1/i111/5870473 (visited on 09/18/2026) (cit. on p. 7).

[OS07] D. Okanohara and K. Sadakane. "Practical entropy-compressed rank/select dictionary". In: *2007 Proceedings of the ninth workshop on algorithm engineering and experiments (ALENEX)*. SIAM. 2007, pp. 60–70 (cit. on pp. 10, 11).

[Pib22] G. E. Pibiri. "Sparse and skew hashing of K-mers". en. In: *Bioinformatics* 38.Supplement_1 (June 2022), pp. i185–i194. URL: https://academic.oup.com/bioinformatics/article/38/Supplement_1/i185/6617506 (visited on 02/25/2026) (cit. on p. 7).

[SMR26] J. Schulte-Mattler and K. Reinert. "RSHash: a fast and space-efficient hash table for k -mers". en. In: *Bioinformatics* 42.Supplement_2 (Aug. 2026), btag440. URL: https://academic.oup.com/bioinformatics/article/doi/10.1093/bioinformatics/btag440/8767308 (visited on 09/15/2026) (cit. on pp. 5, 7–9).